SMART INDIA HACKATHON 2026 · PS 26122

Phase 1 Start Guide

AI-Powered Planning-to-Execution Project Progress Intelligence Platform — Backend

Repository lakshgupta260-dev/sih2k26 · **Stack** FastAPI · PostgreSQL 16 · SQLAlchemy 2 · Alembic · Docker · **Updated** 01 September 2026

Everything you need to get the backend running on your machine and confirm it actually works. Written for someone who has just cloned the repo. **Time to first running server: about 5 minutes with Docker.**

1. What Phase 1 gives you

The skeleton the other ten phases are built on. There is no business logic yet — no login, no projects, no file upload. What exists is the foundation:

Piece	Where	What it does
Application factory	`backend/app/main.py`	Builds the FastAPI app, wires middleware and error handlers
Configuration	`backend/app/core/config.py`	Typed settings from environment; refuses to start with a weak `SECRET_KEY` in production
Database layer	`backend/app/db/`	SQLAlchemy 2.x base, session lifecycle, UUID/timestamp/soft-delete mixins
Migrations	`backend/alembic/`	Alembic reading the DB URL from settings, not from a committed file
Error envelope	`backend/app/core/exceptions.py`	Every failure returns the same JSON shape
Logging	`backend/app/core/logging.py`	Per-request correlation ids, plain or JSON output
Repository base	`backend/app/repositories/base.py`	Generic typed CRUD
Health endpoints	`backend/app/api/v1/health.py`	Liveness and readiness probes
Container setup	`backend/Dockerfile`, `backend/docker-compose.yml`	Postgres 16 + Redis 7 + API
Tests	`backend/tests/`	14 tests, no database required

Two working endpoints. That is the whole API surface right now, and that is expected.

2. Prerequisites

Check what you have:

```
docker --version
python3 --version
```

- **Have Docker?** Use Path A. Nothing else to install.
- **No Docker?** Use Path B. You will need Python 3.11+ and PostgreSQL 14+.

> Anaconda's Python (3.12/3.13) is newer than these dependency pins target. Prefer Docker, or create the venv with an explicit `python3.11`.

3. Path A — Docker (recommended)

Step 1. Confirm the Docker daemon is actually running. `docker --version` succeeds even when it isn't:

```
docker ps
```

If that errors, open Docker Desktop and wait for the whale icon to settle.

Step 2. Create your environment file:

```
cd backend
cp .env.example .env
```

`.env` is gitignored, so it does not exist after a fresh clone, and `docker compose` will not start without it. No edits are needed for local work — compose overrides the database host, and a blank `SECRET_KEY` auto-generates.

Step 3. Build and start:

```
docker compose up --build
```

First run takes 3–6 minutes. Leave this terminal open; it is your live log.

Step 4. Confirm these lines appear:

```
db-1     | database system is ready to accept connections
redis-1  | Ready to accept connections tcp
api-1    | INFO  [alembic.runtime.migration] Context impl PostgresqlImpl
api-1    | application_starting
api-1    | database_connection_ok          <- the one that matters
api-1    | Uvicorn running on http://0.0.0.0:8000
```

You will then see an access-log line every 30 seconds. That is the container healthcheck calling `/api/v1/health` on a loop — it means the probe works.

Step 5. Go to section 5 to verify.

To stop: `Ctrl + C`, then `docker compose down`. Add `-v` only if you want to wipe the database volume as well.

---

4. Path B — Native

Step 1. Install Python 3.11 and PostgreSQL (macOS/Homebrew shown):

```
brew install python@3.11 postgresql@16
brew services start postgresql@16
```

Step 2. Create the database:

```
createdb sih26122
psql -d sih26122 -c "SELECT current_database();"
```

Step 3. Virtual environment and dependencies:

```
cd backend
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Your prompt should now begin with `(.venv)`.

Step 4. Configure:

```
cp .env.example .env
```

Edit `.env` to match your local PostgreSQL. With a Homebrew install your macOS username is usually the superuser and there is no password:

```
POSTGRES_HOST=localhost
POSTGRES_USER=your-mac-username
POSTGRES_PASSWORD=
POSTGRES_DB=sih26122
```

Step 5. Migrate and run:

```
alembic upgrade head
uvicorn app.main:app --reload
```

Step 6. Point VS Code at the venv, or every import shows a red squiggle: `Cmd + Shift + P` → `Python: Select Interpreter` → the entry ending in `backend/.venv/bin/python`.

---

5. Verify it works

Open a **second** terminal — leave the server running in the first.

```
curl -i http://localhost:8000/api/v1/health
curl -i http://localhost:8000/api/v1/health/ready
```

Expected:

```
HTTP/1.1 200 OK
{"status":"ok","environment":"local","version":"0.1.0"}

HTTP/1.1 200 OK
{"status":"ok","database":"up","checks_passed":true}
```

Then open **http://localhost:8000/docs** — Swagger UI, with both endpoints clickable via **Try it out**.

On Docker, also check:

```
docker compose ps      # backend-api-1 should read "Up (healthy)"
```

Run the tests

```
cd backend
source .venv/bin/activate     # Path B only
pytest -v
```

Expect **14 passed**. These need no database, so run them first when something misbehaves — they separate code problems from setup problems.

Prove readiness is honest

The interesting test. Kill the database and watch the app tell the truth:

```
# Docker
docker compose stop db
# Native
brew services stop postgresql@16

curl -i http://localhost:8000/api/v1/health        # 200 - process alive
curl -i http://localhost:8000/api/v1/health/ready  # 503 - "database":"down"
```

Restart it and both return 200 again. A health check that returns 200 no matter what is worse than none, because it hides outages.

---

6. The two endpoints, and why there are two

Endpoint	Question it answers	Touches the DB?
`GET /api/v1/health`	Is the process alive?	No
`GET /api/v1/health/ready`	Can it actually do its job?	Yes — real `SELECT 1`

"The app crashed" and "the app is fine but the database is still booting" need different reactions. Postgres takes a few seconds longer to start than the API. If the API died whenever it could not reach the database, Docker would restart it forever. Instead it stays alive and reports `503 degraded`, so traffic waits. The `depends_on: condition: service_healthy` in `docker-compose.yml` relies on exactly this.

Every response carries an `X-Request-ID` header. Send your own to trace a call through the logs:

```
curl -i -H "X-Request-ID: my-trace-1" http://localhost:8000/api/v1/health
```

7. Project layout

```
backend/app/
├── main.py              application factory, middleware, exception handlers
├── core/                config, constants, errors, logging, middleware
├── db/                  declarative base, mixins, engine and session
├── models/              SQLAlchemy models  (Phase 2+)
├── schemas/             Pydantic v2 request/response contracts
├── repositories/        persistence; owns queries
├── services/            business logic; owns transaction boundaries
├── api/v1/              routers only — no business logic here
├── ai/                  LLM and embedding providers   (Phase 5)
├── ml/                  delay prediction, scikit-learn (Phase 7)
├── document_processing/ PDF / Excel / OCR abstractions (Phase 4)
├── integrations/        Meta WhatsApp, Vapi            (Phase 9-10)
├── notifications/       channel-agnostic dispatch      (Phase 8)
└── utils/
```

Most of those folders are empty placeholders. That is deliberate — the shape is fixed now so later phases drop into place instead of being retrofitted.

The rule that keeps this clean: routers parse and validate, services decide, repositories query. If you find yourself writing a database query in a route handler, it belongs in a repository.

8. Troubleshooting

Symptom	Cause	Fix
`Cannot connect to the Docker daemon`	Docker Desktop not running	Open it, wait for the whale to settle
`port is already allocated`	Something else on 5432	Set `POSTGRES_PORT=5433` in `.env`, re-run
`env file .env not found`	Fresh clone	`cp .env.example .env`
`error parsing value for field "CORS_ORIGINS"`	Malformed list in `.env`	Use `http://a,http://b` — comma-separated, no brackets or quotes
Imports underlined red in VS Code	Interpreter not selected	`Cmd+Shift+P` → Python: Select Interpreter → `.venv`
`command not found: python3.11`	Homebrew did not link it	`/opt/homebrew/bin/python3.11 -m venv .venv`
`Unable to create '.git/index.lock'`	Stale lock from an interrupted git command	`rm -f .git/index.lock`
`database_connection_failed_at_startup`	DB not up yet, or wrong credentials	Check `.env`; on Docker just wait, compose retries
Tests pass but server will not start	Config problem, not code	Read the first traceback line — it is almost always `.env`

9. Command cheat sheet

```
# Docker
docker compose up --build          # build and start everything
docker compose up -d               # start detached
docker compose logs -f api         # follow just the API log
docker compose ps                  # container status and health
docker compose down                # stop and remove
docker compose down -v             # ... and wipe the database volume
docker compose exec api bash       # shell inside the API container

# Database / migrations   (prefix with `docker compose exec api` on Docker)
alembic upgrade head               # apply all migrations
alembic downgrade -1               # roll back one
alembic current                    # what revision is applied
alembic revision --autogenerate -m "add users"

# Tests
pytest -v                          # verbose
pytest tests/test_config.py        # one file
pytest --cov=app                   # with coverage
```

10. What is deliberately not here yet

No authentication, users, projects, file upload, parsing, matching, ML or integrations. No database tables either — `alembic/versions/` is empty, so `alembic upgrade head` is a safe no-op. Phase 2 creates the first real tables and therefore the first migration.

Do not add a dependency to `requirements.txt` before the phase that needs it. The file is ordered by phase for a reason: when a build breaks, the diff points at the phase that broke it.

11. Next

Phase 2 — authentication (register, login, refresh, logout, current user) with access and refresh JWTs and bcrypt hashing; RBAC across `ADMIN`, `PROJECT_MANAGER` and `SITE_SUPERVISOR`; users, projects, memberships and project-level authorization.

Full phase plan: `backend/README.md`.